Who needs a runtime?

I recently read this post by Maincode, found it really interesting, and wanted to do a deeper dive with a focus on building context from smaller steps.

Feel free to give their post a read first!

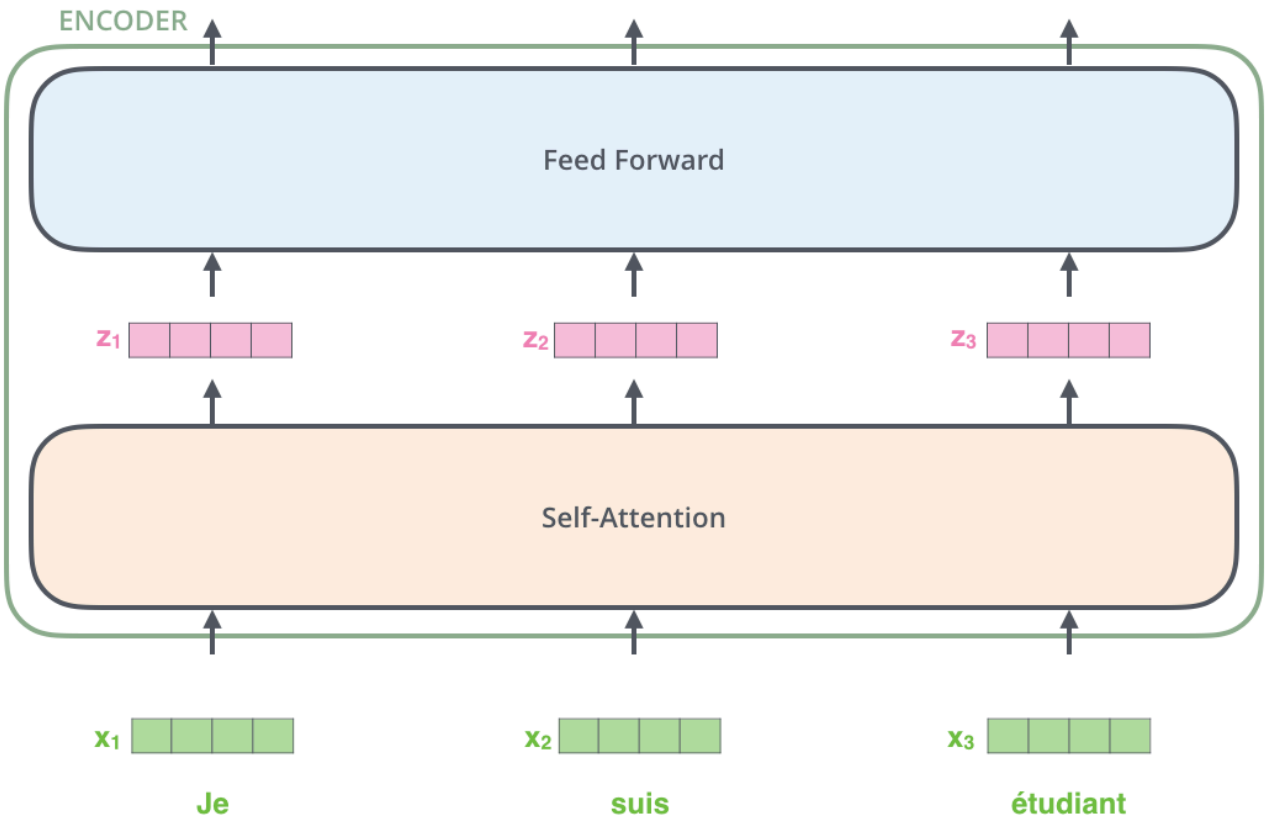
Tracing a token

You write a prompt worth 100 tokens and hit send. Somewhere on the other end, a GPU has to turn that into a response. The two primary kernels responsible for this are prefill and decode.

The prefill is where your 100 input tokens get processed in parallel by computing attention across all of them at once. We then populate the KV cache, which is just a growing pile of already-computed (key, value) pairs that every future token will need to attend to (memoization for the win!). It also "produces the first output token, whose latency is called time-to-first-token (TTFT)." [0]. Prefill is generally compute-bound because the model:

1. Computes Q,K,V for every input token
2. Performs self-attention across the sequence
3. Passes the results through the MLP
4. Stores the resulting K,V pairs in the KV cache.

To mitigate this, techniques like Chunked Prefill are often employed.



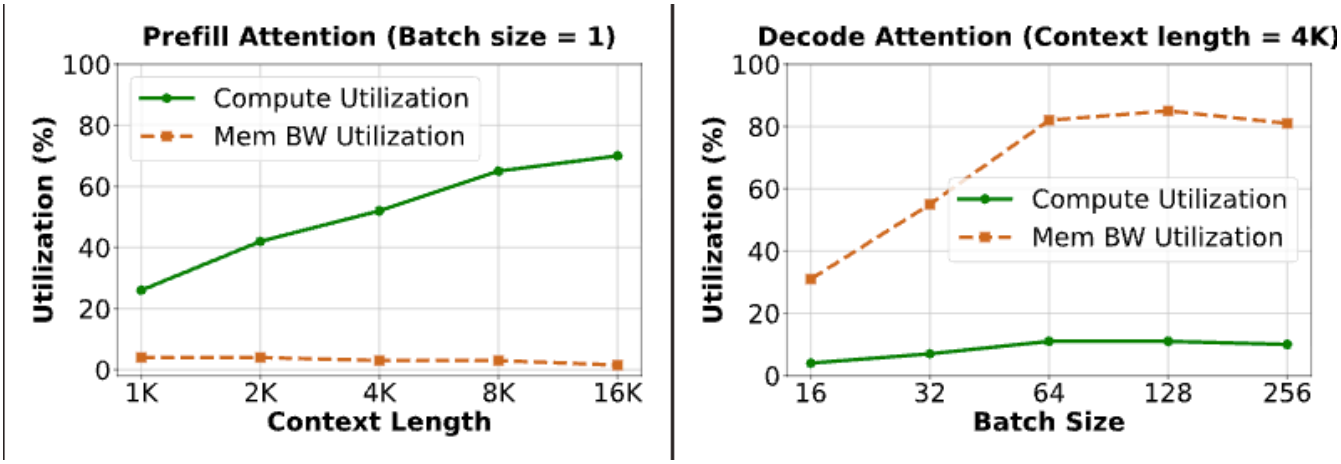
Prefill is naturally a parallel operation. Source: https://jalammar.github.io/illustrated-transformer/

Decoding is where the magic happens.

To generate a new token, the model has to:

1. Load the model weights from HBM
2. Load the entire KV cache (the initial prompt + any previously generated tokens)
3. Compute the new token's q/k/v projections.
4. Run attention against the cached KV
5. Route through the FFN or MoE experts
6. Some kind of softmax to produce a prediction.

Call this an “iteration” [1]. Then it does that again. And again. Decoding results in low compute utilization as it generates one token at a time per request and is generally memory bandwidth bound. (see figure below).



Prefill is compute bound whereas Decode is memory bound. Source: https://arxiv.org/html/2410.18038v2#S1.F1

Now, the weights don't change between iterations and if naively done, they have to be read/streamed for every iteration. The KV cache grows by one entry per token and also has to be re-read in full each time. The activations (essentially the hidden states in between layers) are computed, used, and discarded within a single iteration.

I bring these up because Maincode's post concerns itself with the flow of this data: weights sit still, a cache piles up, and activations vanish. All of them have a different lifespan, bounded by different things, but are all necessary to serve even a single token.

What's in a spec sheet

Maincode deploys on AMD MI355X accelerators; it contains 288 GB of HBM3e and a theoretical peak memory bandwidth of 8 TB/s. Those two may be one of the most important deciding factors as to how soon compute and memory become a limit, respectively.

A bit of math

The Roofline model serves as an effective theoretical framework to assess the potential performance of deploying a model on particular hardware. Every operation has an arithmetic intensity, which is defined as the ratio of compute work, aka FLOPs to data movement (bytes). Now, every piece of hardware has a ridge point: the ratio of its peak compute throughput to its peak memory bandwidth. If your workload's arithmetic intensity falls below the ridge point, you're memory bound; otherwise, compute bound.

Each iteration does roughly 2 FLOPs per parameter (one multiply, one add) and has to load each weight at least once from HBM. So the arithmetic intensity of decode at batch size 1 works out to:

$$\text{arithmetic intensity} = \frac{2 \times N_{\text{params}}}{N_{\text{params}} \times b_{\text{weight}}} = \frac{2}{b_{\text{weight}}}$$

In FP16 that's 1 FLOP/byte. In FP8, 2, and so on and so forth.

Looking at MI355X, the peak FP16 FLOPS figure published for this chip is ~ 2.5 PFLOPs (10^{15}). Against a theoretical bandwidth of 8 TB/s, the ridge point sits at:

$$r = \frac{2500 \text{ TFLOPS}}{8000 \text{ GB/s}} \approx 312 \text{ FLOP/byte}$$

Our decode arithmetic intensity of 2-4 FLOP/byte is not even in the same neighborhood. This means that the compute units are overwhelmingly idle during decode.

This means the speed of token generation is almost entirely set by how fast you can stream bytes out of HBM. A useful approximation from Transformer Inference Arithmetic:

$$t_{\text{token}} \approx \frac{\text{model bytes}}{\text{memory bandwidth}}$$

Say we run Qwen3-235B-A22B, a mixture-of-experts model with 235B total parameters but only ~22B active per token. At FP16 precision, 22B active parameters is roughly 44 GB of weight data per iteration, which spread across 8 GPUs in tensor parallel streams about 5.5 GB per iteration per GPU.

Maincode measured their effective HBM bandwidth ceiling at around 5.8 TB/s per GPU (~72% of the 8 TB/s theoretical peak). At that throughput, streaming 5.5 GB takes roughly 0.95 ms per token, but only if you can keep the memory bandwidth saturated and nothing else gets in the way. In practice, operations like all-reduce (each GPU has a partial result and they all need to exchange and sum so they are on the same page) make it harder. Qwen3-235B-A22B has 94 decoder layers, so that's roughly 188 all-reduces per token generated.

Now Maincode's main (hah) result was that their custom all-reduce uses only

~ 20 µs at the small message sizes that decode uses, compared to ~26 µs for AMD's RCCL at its fastest configuration. This 6 µs gap across those 188 operations compounds to roughly 1.1 ms saved per token. To put that in perspective, that's more than the entire 0.95 ms weight-streaming budget.

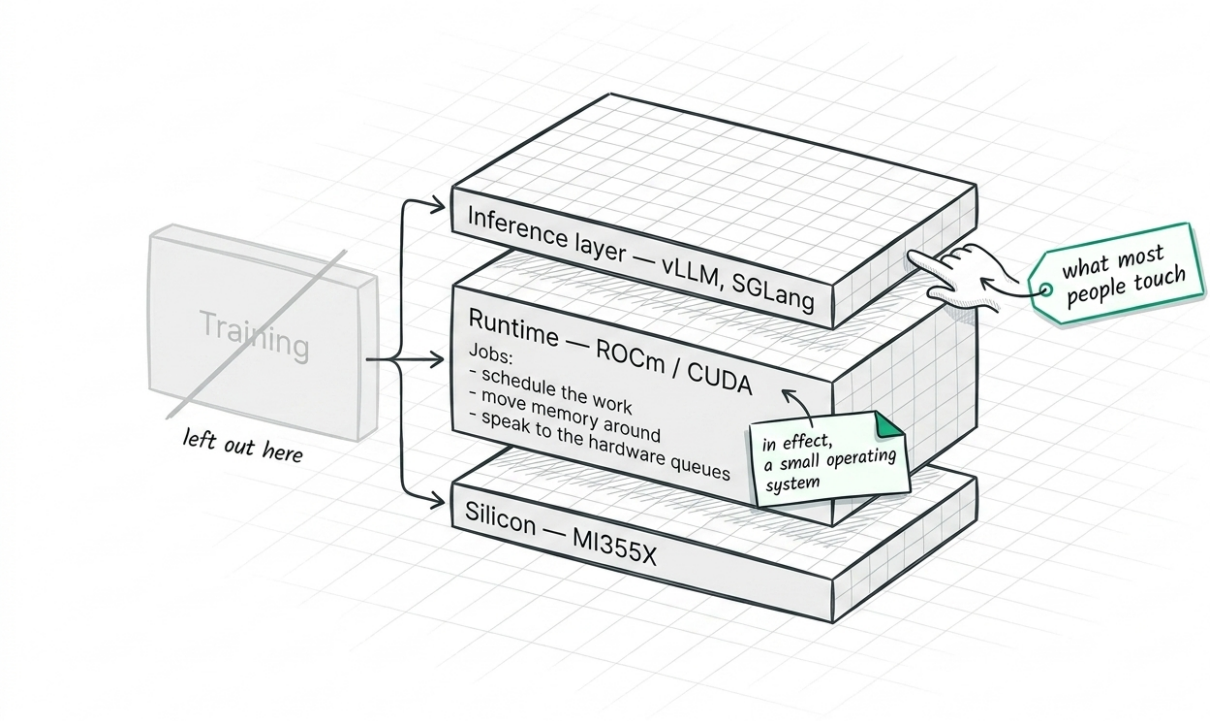
Therefore, the collective communication is the dominant per-layer cost, and I think this is what Maincode meant when they described the all-reduce result as ”the absence of overhead”.

Hopefully it should make a bit more sense now why the notion of completely bypassing the ROCm runtime has some merit.

I should also note that this whole analysis ignores the KV cache read, which at long context lengths becomes the dominant memory cost.

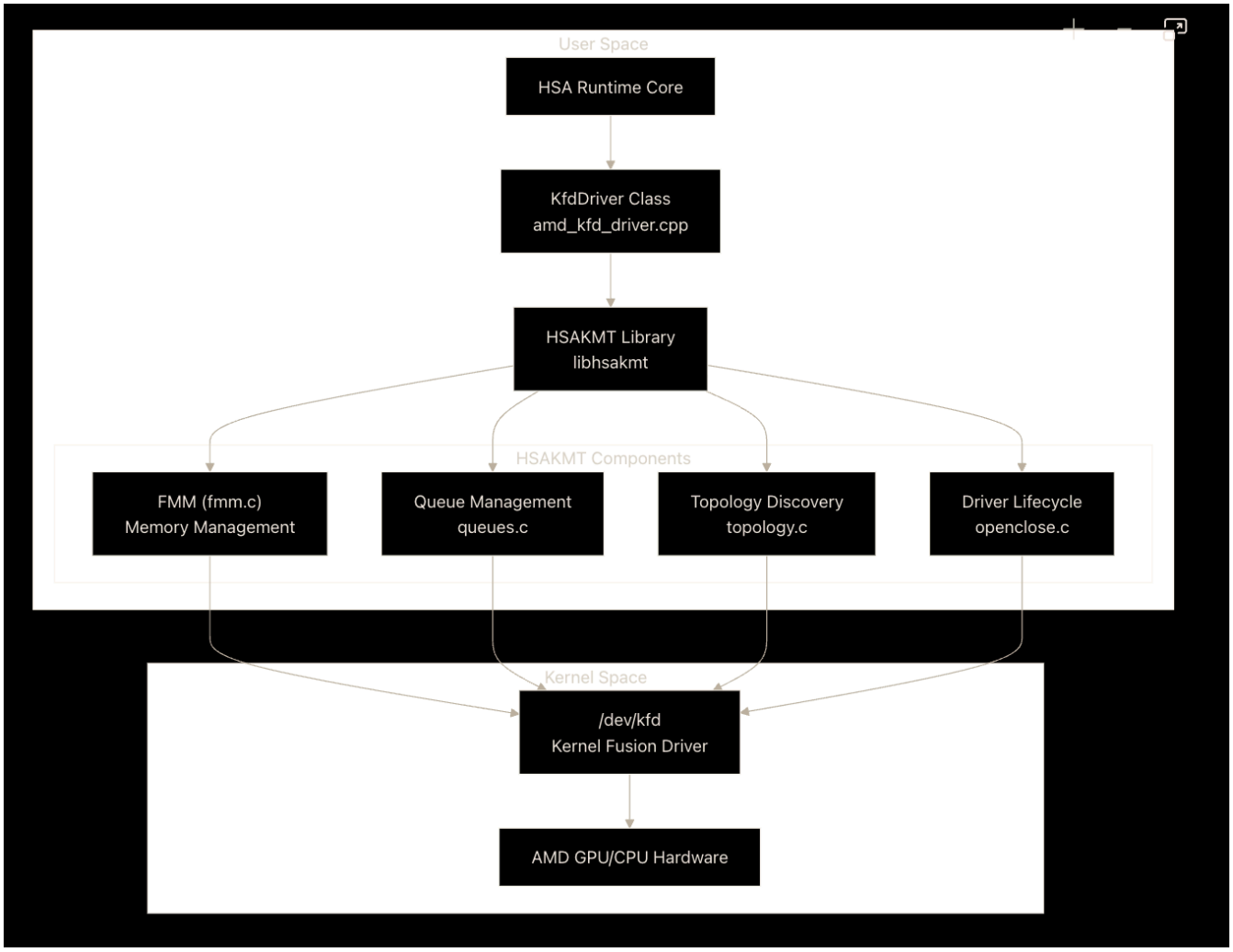
What do you mean by “bypassing the runtime”?

Let's take a second to appreciate what a runtime actually does. It manages command scheduling, memory allocation, synchronization, and everything needed to “accelerate training, fine-tuning, and inference for AI application development.” [2]. This is important to note in the context of Maincode’s post, since by stripping away anything and everything not needed for inference, you could optimize the hot path.



This figure
that was shared in the post is also liable to be misunderstood; you might get the impression that it is the runtime that communicates directly with the silicon, but in fact, that isn't true! What sits in between are kernel drivers.

ROCm is a user-space runtime and it needs kernel-level interfaces that expose hardware capabilities. For AMD, that is provided by Kernel Fusion Driver, and is accessed at */dev/kfd*. Maincode essentially directly communicates with this and "bypasses the runtime". I think the following figure is a helpful visualization.



Source: https://deepwiki.com/ROCm/ROCR-Runtime/4.1-kfd-driver-and-hsakmt

On the fabric asymmetry

The other not-so-intuitive thing I found was that “writing data into a peer GPU streams far better than reading it back out.” I think this contradicts a common mental model where reads are cheap and writes are expensive (partly because they can carry the overhead of consistency guarantees, cache invalidation, etc.).

So, taking a step back, what sort of communication happens inside, say, an 8-GPU MI355X node in the first place?

First, each GPU connects to the host CPU over PCIe, which provides a theoretical peak of 128 GB/s of bidirectional bandwidth. The host dispatches work to the GPUs' command queues and waits for signal completion.

Second, the GPUs connect to each other over AMD's Infinity Fabric using the XGMI protocol. Each MI355X has 7 Infinity Fabric links, one to each of the other 7 GPUs in the node, forming a fully connected mesh. At 153 GB/s per link, this gives each GPU roughly 1 TB/s of total fabric bandwidth to its peers. This is the interconnect that carries the all-reduce traffic we discussed earlier.

Third, each GPU has a NIC connection for multi-node scale-out, used when a model is too large for a single 8-GPU node and you need to communicate across a cluster over the network.

Now consider when GPU 0 wants to share data with GPU 1. GPU 0 can either do a push operation (write the data directly into GPU 1's memory) or GPU 1 can do a pull operation (read from GPU 0's memory). On CPUs, you'd expect these to perform similarly, right?

On GPUs, the opposite holds, for two reinforcing reasons.

First: GPUs use buffered writes. Simply put, small writes get batched into efficient large transfers. Each read, on the other hand, stalls the requesting compute unit until the data actually arrives, and you pay the full round-trip latency of the fabric.

Second: A push completes in half a round trip (GPU 0 writes, it lands in GPU 1's memory, done), whereas a pull requires a full round trip (GPU 1 sends a read request to GPU 0, GPU 0 responds with the data). To note is that the push will need additional buffering for incoming data.

I think this Every µs Matters paper is worth reading for a more rigorous treatment.

Knowing this now, you can design every multi-GPU operation to be latency friendly. Your all-reduce, KV cache distribution or anything that moves data between GPUs should be structured so that the sender writes rather than the receiver reads.

Concluding

There's more in the Maincode post I'd love to dig into, especially how direct chip communication works. Hopefully, if you were newer to inference systems, this gave you enough context to read through their post a little more meaningfully :)

References

[0] POD-Attention: Unlocking Full Prefill-Decode Overlap for Faster LLM Inference

[1] Orca: A Distributed Serving System for Transformer-Based Generative Models

[2] Use ROCm for AI